

ISE 5406: Optimization II

Lecture 13: Conjugate Gradient Methods

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Reading for this lecture

- Chapter 13 of course notes
- Section 8.8 of Bazaraa et al. (2006)
- Chapter 5 of Nocedal and Wright (Conjugate Direction Methods)

Lecture 11: Steepest Descent Method

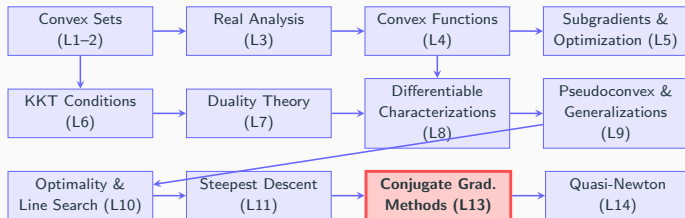
- Gradient-based direction and line search strategy
- Linear convergence rate; slow on ill-conditioned problems
- Simple to implement; requires only gradients

Today (Lecture 13): Conjugate Gradient Methods

- Conjugate directions and their properties
- Conjugate direction method for quadratic programs
- The conjugate gradient (CG) method
- Nonlinear CG methods
- Comparison and practical guidelines

Next (Lecture 14): Quasi-Newton Methods (BFGS & L-BFGS)

Big Picture: Course Roadmap



Why Conjugate Gradient Methods Matter

- Bridge efficiency of Newton's method with simplicity of steepest descent
- CG minimizes quadratics in at most n steps (vs. SD which can take many more)
- Foundation for modern large-scale optimization solvers (L-BFGS, etc.)
- Only requires gradients, no Hessian needed

Newton's Method: Motivation and Basics

- Introduction to Newton's Method

- Newton's Method for Quadratic Programs

- Numerical Example

Convergence Properties

- Convergence Analysis

Line Search and Descent Property

- Using Line Search to Guarantee Descent

Modifications and Extensions

- Levenberg-Marquardt Modification

- Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

Roadmap

Newton's Method: Motivation and Basics

Introduction to Newton's Method

Newton's Method for Quadratic Programs

Numerical Example

Convergence Properties

Convergence Analysis

Line Search and Descent Property

Using Line Search to Guarantee Descent

Modifications and Extensions

Levenberg-Marquardt Modification

Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

Conjugate Directions: Definition

Definition: Given an $n \times n$ **positive definite** matrix Q , the **nonzero** directions $\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_k \in \mathbb{R}^n$ are said to be **Q -conjugate** if

$$\mathbf{d}_i^T Q \mathbf{d}_j = 0 \quad \forall i, j \in \{1, \dots, k\} \text{ with } i \neq j.$$

Intuition: These directions are “orthogonal with respect to the matrix Q ”. They point in directions that are independent according to the geometry defined by Q .

Special Case: If $Q = I$ (identity), then Q -conjugacy reduces to ordinary orthogonality: $\mathbf{d}_i^T \mathbf{d}_j = 0$ for $i \neq j$.

Key Facts:

- We can have **at most n Q -conjugate directions** in $\mathbb{R}^n$
- Any set of Q -conjugate directions are **linearly independent**
- Therefore, n Q -conjugate directions form a **basis for $\mathbb{R}^n$**

Theorem: Linear Independence of Conjugate Directions

Theorem

Let Q be a positive definite $n \times n$ matrix. If $\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_k$ are Q -conjugate directions, then they are **linearly independent**.

Consequence: Since we can have at most n linearly independent directions in $\mathbb{R}^n$, we can have **at most n Q -conjugate directions**.

Further Consequence: Given n Q -conjugate directions $\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_n$, **any vector $\mathbf{x} \in \mathbb{R}^n$ can be written as**

$$\mathbf{x} = \sum_{i=1}^n \alpha_i \mathbf{d}_i \quad \text{for some scalars } \alpha_1, \dots, \alpha_n \in \mathbb{R}.$$

This decomposition is crucial for proving finite convergence.

Proof: Linear Independence of Conjugate Directions

Theorem: If $\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_k$ are Q -conjugate and satisfy $\sum_{i=1}^k \alpha_i \mathbf{d}_i = \mathbf{0}$, then $\alpha_i = 0$ for all i .

Proof:

1. Suppose $\sum_{i=1}^k \alpha_i \mathbf{d}_i = \mathbf{0}$ for some scalars $\alpha_1, \dots, \alpha_k$.
2. Multiply both sides on the left by $\mathbf{d}_j^T Q$:

$$\mathbf{d}_j^T Q \sum_{i=1}^k \alpha_i \mathbf{d}_i = \mathbf{0}.$$

3. By linearity:

$$\sum_{i=1}^k \alpha_i (\mathbf{d}_j^T Q \mathbf{d}_i) = 0.$$

4. Since $\mathbf{d}_1, \dots, \mathbf{d}_k$ are Q -conjugate, we have $\mathbf{d}_j^T Q \mathbf{d}_i = 0$ for all $i \neq j$.
Thus:

$$\alpha_j (\mathbf{d}_j^T Q \mathbf{d}_j) = 0.$$

5. Since Q is positive definite and $\mathbf{d}_j \neq \mathbf{0}$, we have $\mathbf{d}_j^T Q \mathbf{d}_j > 0$.
Therefore, $\alpha_j = 0$.
6. Since j was arbitrary, $\alpha_1 = \alpha_2 = \dots = \alpha_k = 0$. □

Generating Conjugate Directions

Question: Given a positive definite matrix Q , how do we find a set of Q -conjugate directions?

Method 1: Eigendecomposition

- Compute eigenvalues $\lambda_1, \dots, \lambda_n$ and orthonormal eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ of Q
- Set $\mathbf{d}_i = \mathbf{v}_i$ for $i = 1, \dots, n$
- Check: $\mathbf{d}_i^T Q \mathbf{d}_j = \mathbf{v}_i^T Q \mathbf{v}_j = \lambda_j \mathbf{v}_i^T \mathbf{v}_j = 0$ for $i \neq j$ (since eigenvectors are orthonormal)
- **Disadvantage:** Eigendecomposition is expensive for large n

Method 2: Gram-Schmidt Orthogonalization

- Start with any linearly independent vectors $\mathbf{u}_1, \dots, \mathbf{u}_n$
- Apply modified Gram-Schmidt with respect to the Q -inner product

Disadvantage: Requires $O(n^3)$ operations to start with all n vectors

Roadmap

Newton's Method: Motivation and Basics

Introduction to Newton's Method

Newton's Method for Quadratic Programs

Numerical Example

Convergence Properties

Convergence Analysis

Line Search and Descent Property

Using Line Search to Guarantee Descent

Modifications and Extensions

Levenberg-Marquardt Modification

Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

Conjugate Direction Method: Setup and Algorithm

Problem: Minimize the convex quadratic function

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x},$$

where Q is an $n \times n$ **positive definite** matrix and $\mathbf{c} \in \mathbb{R}^n$.

Gradient: $\nabla f(\mathbf{x}) = Q\mathbf{x} + \mathbf{c}$.

Optimal solution: The unique solution is $\mathbf{x}^* = -Q^{-1}\mathbf{c}$.

Algorithm:

- **Given:** Starting point $\mathbf{x}_1 \in \mathbb{R}^n$ and n Q -conjugate directions $\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_n$
- **Iterate:** For $k = 1, 2, \dots, n$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k,$$

where $\lambda_k = \arg \min_{\lambda \in \mathbb{R}} f(\mathbf{x}_k + \lambda \mathbf{d}_k)$ (exact line search)

- **Convergence:** This method converges to $\mathbf{x}^*$ in at most n iterations

Exact Step Size Formula

For the quadratic function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$, minimize along direction $\mathbf{d}_k$:

$$\theta_k(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{d}_k).$$

Expanding:

$$\begin{aligned}\theta_k(\lambda) &= \frac{1}{2}(\mathbf{x}_k + \lambda \mathbf{d}_k)^T Q (\mathbf{x}_k + \lambda \mathbf{d}_k) + \mathbf{c}^T (\mathbf{x}_k + \lambda \mathbf{d}_k) \\ &= \frac{1}{2} \mathbf{d}_k^T Q \mathbf{d}_k \cdot \lambda^2 + (Q \mathbf{x}_k + \mathbf{c})^T \mathbf{d}_k \cdot \lambda + \text{const} \\ &= \frac{1}{2} \mathbf{d}_k^T Q \mathbf{d}_k \cdot \lambda^2 + \nabla_k^T \mathbf{d}_k \cdot \lambda + \text{const},\end{aligned}$$

where $\nabla_k = Q \mathbf{x}_k + \mathbf{c} = \nabla f(\mathbf{x}_k)$.

Setting $\frac{d\theta_k}{d\lambda} = 0$:

$$\mathbf{d}_k^T Q \mathbf{d}_k \cdot \lambda + \nabla_k^T \mathbf{d}_k = 0,$$

giving

$$\lambda_k = -\frac{\nabla_k^T \mathbf{d}_k}{\mathbf{d}_k^T Q \mathbf{d}_k}.$$

Property 1: Orthogonal Gradients

Theorem (Orthogonal Gradients): For the conjugate direction method,

$$\nabla_{k+1}^T \mathbf{d}_i = 0 \quad \forall i \in \{1, 2, \dots, k\}.$$

In particular, $\nabla_{k+1}^T \mathbf{d}_k = 0$ for all k (gradient orthogonal to search direction).

Proof (for $\nabla_{k+1}^T \mathbf{d}_k = 0$):

1. The step size λ_k minimizes $\theta_k(\lambda)$, so:

$$\left. \frac{d\theta_k}{d\lambda} \right|_{\lambda=\lambda_k} = 0 \quad \Rightarrow \quad \nabla_k^T \mathbf{d}_k + \lambda_k \mathbf{d}_k^T \mathbf{Q} \mathbf{d}_k = 0.$$

2. The updated gradient is $\nabla_{k+1} = \nabla_k + \lambda_k \mathbf{Q} \mathbf{d}_k$. Thus:

$$\nabla_{k+1}^T \mathbf{d}_k = (\nabla_k + \lambda_k \mathbf{Q} \mathbf{d}_k)^T \mathbf{d}_k = \nabla_k^T \mathbf{d}_k + \lambda_k \mathbf{d}_k^T \mathbf{Q} \mathbf{d}_k = 0. \quad \square$$

Consequence: The gradient is orthogonal to the direction we just moved in. This means we won't undo progress made in the previous step.

Property 2: Expanding Subspace Theorem

Theorem (Expanding Subspace): After k iterations of the conjugate direction method, the point $\mathbf{x}_{k+1}$ minimizes f over the affine subspace

$$\mathbf{x}_1 + \text{span}(\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_k) = \left\{ \mathbf{x}_1 + \sum_{i=1}^k \alpha_i \mathbf{d}_i : \alpha_1, \dots, \alpha_k \in \mathbb{R} \right\}.$$

Proof Sketch:

- Define $\theta_k(\boldsymbol{\alpha}) = f(\mathbf{x}_1 + D^k \boldsymbol{\alpha})$ where $D^k = [\mathbf{d}_1 \mid \dots \mid \mathbf{d}_k]$ and $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_k)^T$.
- The gradient with respect to $\boldsymbol{\alpha}$ is:

$$\nabla_{\boldsymbol{\alpha}} \theta_k = D^{kT} \nabla f(\mathbf{x}_1 + D^k \boldsymbol{\alpha}).$$

- After k iterations with exact line search along $\mathbf{d}_1, \dots, \mathbf{d}_k$:

$$\nabla_{\boldsymbol{\alpha}} \theta_k(\boldsymbol{\lambda}^k) = D^{kT} \nabla_{k+1} = [\nabla_{k+1}^T \mathbf{d}_1, \dots, \nabla_{k+1}^T \mathbf{d}_k]^T = \mathbf{0}$$

by the orthogonal gradients property. Thus $\boldsymbol{\lambda}^k$ minimizes θ_k . □

Convergence of the Conjugate Direction Method

Theorem

For a positive definite quadratic function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$, the conjugate direction method with n Q -conjugate directions **converges to the global minimizer $\mathbf{x}^*$ in at most n steps**, regardless of starting point.

Proof:

- Since $\mathbf{d}_1, \dots, \mathbf{d}_n$ are Q -conjugate, they are linearly independent and form a basis for $\mathbb{R}^n$.
- Any vector $\mathbf{x} \in \mathbb{R}^n$ can be written as $\mathbf{x} = \mathbf{x}_1 + \sum_{i=1}^n \alpha_i \mathbf{d}_i$.
- Minimizing f over $\mathbb{R}^n$ is equivalent to minimizing $\theta_n(\boldsymbol{\alpha}) = f(\mathbf{x}_1 + D^n \boldsymbol{\alpha})$ over all $\boldsymbol{\alpha} \in \mathbb{R}^n$.
- By the expanding subspace theorem, after n iterations, $\mathbf{x}_{n+1}$ minimizes θ_n , hence minimizes f globally.
- Therefore, $\mathbf{x}_{n+1} = \mathbf{x}^*$. □

Numerical Example: Convergence Comparison

Example: Minimize

$$f(x_1, x_2) = 4x_1^2 + 4x_2^2 - 4x_1x_2 - 12x_2.$$

Matrix form: $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$ where

$$Q = \begin{pmatrix} 8 & -4 \\ -4 & 8 \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} 0 \\ -12 \end{pmatrix}.$$

Optimal solution: $\mathbf{x}^* = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$

Convergence Behavior:

- **Steepest Descent:** Zigzag pattern toward optimum; many iterations
- **Conjugate Gradient:** Smooth path; converges in exactly 2 iterations (since $n = 2$)

This example illustrates the key advantage: CG reaches the solution in n

Roadmap

Newton's Method: Motivation and Basics

Introduction to Newton's Method

Newton's Method for Quadratic Programs

Numerical Example

Convergence Properties

Convergence Analysis

Line Search and Descent Property

Using Line Search to Guarantee Descent

Modifications and Extensions

Levenberg-Marquardt Modification

Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

The Conjugate Gradient Method: Motivation

Challenge with Conjugate Direction Method:

- Requires knowing n conjugate directions **in advance**
- Computing all n directions is expensive: $O(n^3)$ via Gram-Schmidt
- Storing all n directions requires $O(n^2)$ memory

Solution: Conjugate Gradient (CG) Method

- Generates conjugate directions **on the fly** while optimizing
- Uses a **three-term recurrence**:

$$\mathbf{d}_{k+1} = -\nabla_{k+1} + \beta_k \mathbf{d}_k$$

- Avoids storing all previous directions (only store current direction)
- Maintains Q -conjugacy automatically

Key Insight: Each new direction is a combination of:

Computing β_k for Q-Conjugacy

Goal: Choose β_k so that $\mathbf{d}_{k+1}^T Q \mathbf{d}_j = 0$ for all $j = 1, 2, \dots, k$.

Key Lemma: It suffices to have $\mathbf{d}_{k+1}^T Q \mathbf{d}_k = 0$ (conjugacy with just the previous direction).

This is due to the special structure of the CG recurrence and the orthogonal gradient property.

Derivation:

$$\begin{aligned}\mathbf{d}_{k+1}^T Q \mathbf{d}_k &= (-\nabla_{k+1} + \beta_k \mathbf{d}_k)^T Q \mathbf{d}_k \\ &= -\nabla_{k+1}^T Q \mathbf{d}_k + \beta_k \mathbf{d}_k^T Q \mathbf{d}_k = 0.\end{aligned}$$

Solving for β_k :

$$\beta_k = \frac{\nabla_{k+1}^T Q \mathbf{d}_k}{\mathbf{d}_k^T Q \mathbf{d}_k}.$$

Note: This requires multiplying the gradient by Q , which is practical only if we can compute Hessian-vector products or if we have the matrix Q explicitly (as in quadratic problems).

The Conjugate Gradient Algorithm

Input: Starting point $\mathbf{x}_1 \in \mathbb{R}^n$, matrix Q , vector $\mathbf{c}$

Output: Solution to $\min f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$

1. Compute $\nabla_1 = Q\mathbf{x}_1 + \mathbf{c}$
2. Set $\mathbf{d}_1 = -\nabla_1$
3. **For** $k = 1, 2, \dots, n - 1$:
 - 3.1 Compute step size: $\lambda_k = -\frac{\nabla_k^T \mathbf{d}_k}{\mathbf{d}_k^T Q \mathbf{d}_k}$
 - 3.2 Update iterate: $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$
 - 3.3 Compute gradient: $\nabla_{k+1} = \nabla_k + \lambda_k Q \mathbf{d}_k$ (cheaper than recomputing from scratch)
 - 3.4 Compute coefficient: $\beta_k = \frac{\nabla_{k+1}^T Q \mathbf{d}_k}{\mathbf{d}_k^T Q \mathbf{d}_k}$
 - 3.5 Update direction: $\mathbf{d}_{k+1} = -\nabla_{k+1} + \beta_k \mathbf{d}_k$
4. Return $\mathbf{x}_{n+1}$ or check earlier termination

Convergence of the CG Method

Theorem

The CG method produces n Q -conjugate directions $\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_n$.

Consequently, **CG converges to the global minimum in at most n steps** for any convex quadratic problem (in exact arithmetic).

Why CG is Superior to Steepest Descent:

- **Steepest Descent:** Converges linearly; may take $\sim \kappa(Q)$ iterations where κ is the condition number
- **CG Method:** Converges in exactly n steps (finite termination), regardless of condition number

Computational Cost:

- Cost per iteration: $O(n^2)$ (one matrix-vector product with Q)
- Total cost for n iterations: $O(n^3)$
- Storage: $O(n)$ (only current iterate, gradient, direction)

Exercise 1: Verifying Q -Conjugacy

Quick Check

Let $Q = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ and consider the directions $\mathbf{d}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\mathbf{d}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

Are $\mathbf{d}_1$ and $\mathbf{d}_2$ Q -conjugate? Check by computing $\mathbf{d}_1^T Q \mathbf{d}_2$.

Solution 1: Verifying Q -Conjugacy

Answer

Check whether $\mathbf{d}_1^T Q \mathbf{d}_2 = 0$:

$$\mathbf{d}_1^T Q \mathbf{d}_2 = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 0.$$

Also, $\mathbf{d}_2^T Q \mathbf{d}_1 = 0$ by symmetry of Q .

Conclusion: Yes, $\mathbf{d}_1$ and $\mathbf{d}_2$ are Q -conjugate because $\mathbf{d}_1^T Q \mathbf{d}_2 = 0$.

Remark: For this diagonal matrix Q , the eigenvectors are the coordinate axes, so they are automatically conjugate with respect to Q .

Exercise 2: CG on a 2×2 Problem

Computational Exercise

Apply the conjugate gradient method to minimize

$$f(x_1, x_2) = x_1^2 + 4x_2^2 - 2x_1 - 4x_2,$$

starting from $\mathbf{x}_1 = (0, 0)^T$.

Steps:

- Write f in the form $\frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$
- Compute initial gradient ∇_1 and direction $\mathbf{d}_1$
- Perform one CG iteration to find $\mathbf{x}_2$

Solution 2: CG on a 2×2 Problem (Part 1)

Answers

(1) Matrix Form:

$$f(\mathbf{x}) = x_1^2 + 4x_2^2 - 2x_1 - 4x_2 = \frac{1}{2} \mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x} + \text{const}$$

$$\text{where } Q = \begin{pmatrix} 2 & 0 \\ 0 & 8 \end{pmatrix} \text{ and } \mathbf{c} = \begin{pmatrix} -2 \\ -4 \end{pmatrix}.$$

(2) Initial Gradient:

$$\nabla_1 = Q\mathbf{x}_1 + \mathbf{c} = \begin{pmatrix} 2 & 0 \\ 0 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} -2 \\ -4 \end{pmatrix} = \begin{pmatrix} -2 \\ -4 \end{pmatrix}.$$

(3) First Direction:

$$\mathbf{d}_1 = -\nabla_1 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}.$$

(4) Step Size:

$$\nabla_1^T \mathbf{d}_1 = (-2)(2) + (-4)(4) = -4 - 16 = -20.$$

$$\mathbf{d}_1^T Q \mathbf{d}_1 = (2)(4) + (4)(32) = 8 + 128 = 136.$$

$$\lambda_1 = -\frac{-20}{136} = \frac{20}{136} = \frac{5}{34}.$$

Solution 2: CG on a 2×2 Problem (Part 2)

Answers (continued)

(5) Updated Iterate:

$$\mathbf{x}_2 = \mathbf{x}_1 + \lambda_1 \mathbf{d}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{5}{34} \begin{pmatrix} 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 10/34 \\ 20/34 \end{pmatrix} = \begin{pmatrix} 5/17 \\ 10/17 \end{pmatrix}.$$

(6) Updated Gradient:

$$\begin{aligned} \nabla_2 &= Q\mathbf{x}_2 + \mathbf{c} = \begin{pmatrix} 2 & 0 \\ 0 & 8 \end{pmatrix} \begin{pmatrix} 5/17 \\ 10/17 \end{pmatrix} + \begin{pmatrix} -2 \\ -4 \end{pmatrix} \\ &= \begin{pmatrix} 10/17 \\ 80/17 \end{pmatrix} + \begin{pmatrix} -34/17 \\ -68/17 \end{pmatrix} = \begin{pmatrix} -24/17 \\ 12/17 \end{pmatrix}. \end{aligned}$$

(7) Conjugacy Coefficient:

$$\begin{aligned} \nabla_2^T Q \mathbf{d}_1 &= \begin{pmatrix} -24/17 \\ 12/17 \end{pmatrix}^T \begin{pmatrix} 4 \\ 32 \end{pmatrix} = \frac{-96 + 384}{17} = \frac{288}{17}. \\ \beta_1 &= \frac{288/17}{136} \approx 0.124. \end{aligned}$$

For a 2×2 problem, the next iteration would reach the exact solution $\mathbf{x}^* = (1, 1)^T$.

Roadmap

Newton's Method: Motivation and Basics

Introduction to Newton's Method

Newton's Method for Quadratic Programs

Numerical Example

Convergence Properties

Convergence Analysis

Line Search and Descent Property

Using Line Search to Guarantee Descent

Modifications and Extensions

Levenberg-Marquardt Modification

Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

Nonlinear Conjugate Gradient Methods

Challenge: CG was designed for quadratic functions. What about **general nonlinear** functions?

Idea: Extend CG by:

1. Using **gradients only** (no Hessian Q needed)
2. Replacing **exact minimization** with **inexact line search** (Wolfe conditions)
3. Introducing **restart strategies** to handle non-quadratic behavior
4. Using **alternative β_k formulas** that don't require Q

Direction Formula: Still use the recurrence

$$\mathbf{d}_{k+1} = -\nabla_{k+1} + \beta_k \mathbf{d}_k,$$

but now:

- β_k is computed using gradient information only

Formulas for β_k in Nonlinear CG

For quadratic problems with exact line search, all formulas give the same β_k . For nonlinear functions, they differ:

Fletcher-Reeves (FR) Formula

$$\beta_k^{\text{FR}} = \frac{\nabla_{k+1}^T \nabla_{k+1}}{\nabla_k^T \nabla_k}.$$

- Uses only gradients (simplest to implement)
- Mathematically equivalent to original for quadratic with exact line search
- Can be sensitive to roundoff errors in practice

Polak-Ribière-Polyak (PRP) Formula

$$\beta_k^{\text{PRP}} = \frac{\nabla_{k+1}^T (\nabla_{k+1} - \nabla_k)}{\nabla_k^T (\nabla_k - \nabla_{k-1})}.$$

Nonlinear Conjugate Gradient Algorithm

Input: Function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, starting point $\mathbf{x}_1 \in \mathbb{R}^n$, tolerance $\epsilon > 0$

Output: Approximate minimizer $\mathbf{x}$

1. Compute $\nabla_1 = \nabla f(\mathbf{x}_1)$
2. Set $\mathbf{d}_1 = -\nabla_1$
3. Set $k = 1$
4. **While** $\|\nabla_k\| > \epsilon$ **and** $k < k_{\max}$:
 - 4.1 (Wolfe Line Search) Find $\lambda_k > 0$ satisfying:
 - $f(\mathbf{x}_k + \lambda_k \mathbf{d}_k) \leq f(\mathbf{x}_k) + c_1 \lambda_k \nabla_k^T \mathbf{d}_k$ (sufficient decrease)
 - $|\nabla(\mathbf{x}_k + \lambda_k \mathbf{d}_k)^T \mathbf{d}_k| \leq c_2 |\nabla_k^T \mathbf{d}_k|$ (curvature condition)
 - 4.2 Update iterate: $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$
 - 4.3 Compute gradient: $\nabla_{k+1} = \nabla f(\mathbf{x}_{k+1})$
 - 4.4 Compute β_k using FR, PRP, or similar formula
 - 4.5 Update direction: $\mathbf{d}_{k+1} = -\nabla_{k+1} + \beta_k \mathbf{d}_k$
 - 4.6 (Optional Restart) If $\|\nabla_{k+1}\|$ not decreasing or $k \bmod n = 0$: set $\mathbf{d}_{k+1} = -\nabla_{k+1}$
 - 4.7 Set $k = k + 1$
5. Return $\mathbf{x}_k$

Key differences from quadratic CG:

- Wolfe line search instead of exact minimization
- Optional periodic restart every n iterations or when progress stalls
- No guaranteed finite convergence (but fast in practice)

```
frametitlefg=white,bg=orange!70!black
```

Exercise 3: Nonlinear CG on Rosenbrock Function

Analysis Exercise

Consider the Rosenbrock function:

$$f(x_1, x_2) = x_1^2 + 100(x_2 - x_1^2)^2.$$

Starting from $\mathbf{x}_1 = (-1.2, 1)^T$ (standard Rosenbrock start):

1. Compute the gradient $\nabla f(\mathbf{x}_1)$
2. Set the first CG direction $\mathbf{d}_1 = -\nabla_1$
3. Explain why nonlinear CG performs better than steepest descent on this function

Solution 3: Nonlinear CG on Rosenbrock

Answers

(1) Gradient at Start:

$$\nabla f(x_1, x_2) = \begin{pmatrix} 2x_1 - 400x_1(x_2 - x_1^2) \\ 200(x_2 - x_1^2) \end{pmatrix}.$$

At $\mathbf{x}_1 = (-1.2, 1)^T$:

$$\nabla_1 = \begin{pmatrix} 2(-1.2) - 400(-1.2)(1 - 1.44) \\ 200(1 - 1.44) \end{pmatrix} = \begin{pmatrix} -2.4 + 211.2 \\ -88 \end{pmatrix} = \begin{pmatrix} 208.8 \\ -88 \end{pmatrix}.$$

(2) First Direction:

$$\mathbf{d}_1 = -\nabla_1 = \begin{pmatrix} -208.8 \\ 88 \end{pmatrix}.$$

(3) Why CG Beats Steepest Descent:

- Rosenbrock has a **long, narrow valley** (very ill-conditioned)
- **Steepest descent** zigzags across the valley, requiring many iterations
- **CG method** uses information from previous iterations to choose directions that avoid zigzagging
- CG takes larger, more productive steps toward the narrow valley floor

Roadmap

Newton's Method: Motivation and Basics

Introduction to Newton's Method

Newton's Method for Quadratic Programs

Numerical Example

Convergence Properties

Convergence Analysis

Line Search and Descent Property

Using Line Search to Guarantee Descent

Modifications and Extensions

Levenberg-Marquardt Modification

Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

Comparison: Steepest Descent, Newton, and CG

Property	Steepest Descent	Newton	Conjugate Gradient
Cost/iteration	$O(n^2)$	$O(n^3)$	$O(n^2)$
Storage	$O(n)$	$O(n^2)$	$O(n)$
Conv. (convex quad.)	Linear, slow	Quadratic	Exact in $\leq n$
Conv. (smooth NL)	Linear	Quadratic (local)	Superlinear
Requires Hessian?	No	Yes	No
Robust to ill-cond.?	Poor	Good	Good
Easy to implement?	Very easy	Moderate	Easy
Large n (millions)?	Yes	No	Yes

Summary:

- **Steepest Descent:** Simple and robust, but slow on ill-conditioned problems
- **Newton:** Fast, but expensive ($O(n^3)$ per step) and requires Hessian
- **Conjugate Gradient:** Best compromise: efficient, no Hessian, superlinear convergence

Practical Guidelines: Choosing Your Algorithm

Use Steepest Descent when:

- Simple, robust method needed with minimal implementation
- Problem is well-conditioned (small condition number κ)
- Quick sanity check or prototype
- No structure available for exploitation

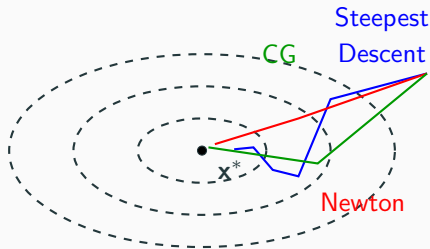
Use Newton's Method when:

- Can cheaply compute/access the Hessian matrix
- Need superlinear/quadratic final convergence
- Problem is locally strongly convex
- Computational budget allows $O(n^3)$ per iteration (small n)

Use Conjugate Gradient when:

Convergence Paths: Visual Comparison

For a strongly convex quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}$:



Observations:

- **Steepest Descent**: Zigzag; many iterations proportional to $\kappa(Q)$
- **Newton**: 1-2 iterations but expensive per step
- **Conjugate Gradient**: Smooth path; converges in n exact steps

```
frametitlefg=white,bg=orange!70!black
```

Exercise 4: Choosing the Right Optimization Method

Decision Exercise

For each scenario, choose the best method (SD, Newton, or CG) and justify:

1. Minimize $f(\mathbf{x}) = \frac{1}{2}\|\mathbf{x} - \mathbf{b}\|^2$ subject to linear constraints, $n = 10^6$
2. Minimize a convex quadratic with sparse Q , $n = 100$
3. Minimize a smooth nonlinear function, $n = 50$, dense Hessian
4. Train a neural network ($n = 10^7$, nonconvex loss)

Solution 4: Method Selection

Answers

(1) **CG** is best. Very large-scale constrained least squares solved using CG with projection. $O(n)$ memory, no Hessian needed.

(2) **CG** is best. Quadratic with sparse matrix—matrix-vector products are cheap. CG converges in ≤ 100 steps exactly. Newton would require expensive Cholesky factorization.

(3) **CG** is best. Smooth nonlinear, moderate n , no easy Hessian. CG provides superlinear convergence without $O(n^3)$ per-iteration cost of Newton.

(4) **L-BFGS** (advanced method, next lecture concept). Very large nonconvex; CG assumes convexity. Modern deep learning uses first-order methods: SD with momentum, Adam, or L-BFGS variants.

Insight: For most practical optimization, especially large-scale, CG or quasi-Newton (L-BFGS) is the go-to choice.

Roadmap

Newton's Method: Motivation and Basics

Introduction to Newton's Method

Newton's Method for Quadratic Programs

Numerical Example

Convergence Properties

Convergence Analysis

Line Search and Descent Property

Using Line Search to Guarantee Descent

Modifications and Extensions

Levenberg-Marquardt Modification

Quasi-Newton Methods

Summary and Key Takeaways

Looking Ahead

Summary: Key Concepts from Lecture 13

Core Ideas:

- **Conjugate directions** are orthogonal with respect to matrix Q ; form a powerful basis
- **Conjugate direction method** minimizes convex quadratics in $\leq n$ steps using exact line search
- **Expanding subspace theorem** explains convergence: after k iterations, we've minimized over the subspace spanned by the first k directions
- **Conjugate gradient method** generates directions automatically via three-term recurrence, avoiding full storage

Main Results:

- **CG converges in at most n steps** for quadratic functions (exact arithmetic)

Next Topic: Quasi-Newton Methods (BFGS & L-BFGS)

Motivation:

- Newton's method is fast but requires expensive Hessian computation
- Can we approximate the Hessian without computing it explicitly?
- Use low-rank updates to maintain superlinear convergence

Topics to Come:

- Secant conditions and Hessian approximation
- BFGS update (Broyden-Fletcher-Goldfarb-Shanno)
- L-BFGS (Limited-memory BFGS) for very large-scale problems
- Convergence analysis and practical implementation
- Comparison with CG and Newton

Exercises Summary and Further Practice

Exercises Covered in This Lecture:

1. [Exercise 1](#): Verifying Q -conjugacy of given directions
2. [Exercise 2](#): Running CG algorithm by hand on a 2×2 quadratic
3. [Exercise 3](#): Understanding nonlinear CG on the Rosenbrock function
4. [Exercise 4](#): Selecting the right optimization method for various scenarios

Recommended Practice:

- Implement basic quadratic CG in Python or MATLAB
- Verify finite convergence in n steps on test quadratics
- Try nonlinear CG on simple functions (e.g., Rosenbrock, Beale)
- Compare convergence behavior of SD, CG, and Newton on ill-conditioned problems
- Explore effect of different β formulas (FR vs. PRP)

Questions?

Lecture 13: Conjugate Gradient Methods

ISE 5406: Optimization II

Dr. Robert Hildebrand